

Getting Started for Application Developers

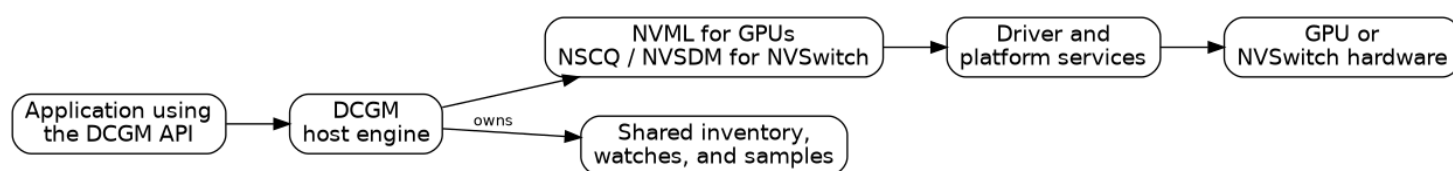
DCGM gives applications a common model for NVIDIA hardware and a long-lived service for collecting management data. Clients share watches and retained samples instead of building independent polling loops.

A deployed application cannot know which monitors, schedulers, exporters, support tools, or co-scheduled workloads will query the same hardware later. The host engine coordinates DCGM collection without requiring those clients to know about one another.

The Low-Level Software Stack

The NVIDIA driver owns and controls a GPU. NVIDIA Management Library (NVML) is the stable C management interface supplied with the driver.

DCGM consumes NVML for many GPU operations. For NVSwitch monitoring, DCGM uses generation-appropriate platform libraries such as NSCQ or NVSDM. Drivers and fabric services remain responsible for the hardware; Fabric Manager, where required, configures and manages the fabric. DCGM clients use one entity and field model across these provider-specific layers.



DCGM over device-specific management software

Why DCGM Is Stateful

Without a shared collection service, every application interested in GPU power would need its own polling schedule. An exporter or scheduler interested in the same value would create another schedule and another series of calls.

Those calls are not necessarily cheap reads from process memory. Many cross into the driver, and some require device or firmware work. Application developers cannot coordinate their polling with every management consumer that a site might deploy.

DCGM moves that coordination into the host engine. A client requests a *watch* for an entity-field pair and supplies an update interval and retention limits. The engine combines compatible requests into one collection stream:

Client request	Effective host-engine behavior
Update interval	Collect at the fastest interval required by an active watcher.
Retention	Retain enough history for the longest active requirement.
Additional reader	Serve retained samples without adding another underlying collection loop when the existing watch already satisfies it.
Removed watch	Continue collection while another watcher still requires the entity-field pair.

Each sample records the entity, field, collection timestamp, value, and status. Reading the latest value normally reads this cache; it does not force a new low-level call. The first sample might not exist until an update cycle has completed.

Retention makes more than efficient polling possible. Clients can retrieve a time window, preserve a sparse event until it is consumed, calculate a counter delta, or summarize a process or job without having observed every sample in real time.

Stateful collection is not free. Watching more fields or entities, collecting faster, and retaining longer histories increase host-engine work and memory use. The scaling benefit is that adding a compatible reader does not normally duplicate the underlying device collection.

Only clients of the same host engine share this state. Separate embedded host engines have separate watches and caches, and starting DCGM only for an individual query discards the history and sharing that justify using it.

What Applications Gain from DCGM

DCGM application services

Service	Application benefit
Shared collection	Compatible clients use one effective watch and one retained sample stream for an entity-field pair.
Common entity model	GPUs, MIG instances, NVSwitches, links, supported CPUs, and supported adapters use consistent identity and grouping concepts.
Typed fields	Telemetry, counters, events, and inventory values include metadata, timestamps, and status.

Service	Application benefit
Retained history	Applications can retrieve time windows and calculate interval results without receiving every sample in real time.
Higher-level workflows	Health, policy, configuration enforcement, diagnostics, profiling, and process or job summaries build on the same host state.
Operational interoperability	API clients, command-line tools, exporters, and schedulers can inspect the same engine and refer to the same persistent groups.

DCGM defines its own field and entity semantics rather than mirroring each provider-specific interface. Applications should discover entity IDs and field support at runtime and preserve the status and timestamp returned with each value.

Make availability of a DCGM endpoint part of the application's deployment contract. Ordinary workloads should use the endpoint supplied by the platform instead of starting private engines with independent caches and watch schedules.

Choose a Host-Engine Deployment

An ordinary workload should use the DCGM endpoint provided by the platform. Embedding DCGM is for a designated node management agent that owns the engine's complete lifecycle.

Standalone host engine

In standalone operation, `nv-hostengine` owns DCGM. API clients, operator tools, exporters, and schedulers can connect to it and share runtime state.

Choose standalone operation when multiple consumers are known or possible, the platform owns service lifetime and privileges, or operators require `dcgmi`. If exclusive ownership cannot be guaranteed for the complete deployment lifetime, use standalone operation.

Treat endpoint address, access policy, supported DCGM version, and restart behavior as part of the deployment contract. See [client-access security](#).

Embedded host engine

In embedded operation, a long-running management agent loads `libdcgm.so` and starts the host engine inside its process. The agent owns initialization, shutdown, credentials, state recreation, update scheduling, and failure handling.

Use embedded operation only when the platform design assigns that ownership to the application. Separate embedded engines do not share caches or watch schedules.

Embedded and manual are separate choices. `dcgmStartEmbedded()` accepts:

`DCGM_OPERATION_MODE_AUTO`

DCGM runs the background work required by watched fields and policies.

`DCGM_OPERATION_MODE_MANUAL`

The embedding agent must call APIs such as `dcgmUpdateAllFields()` and `dcgmPolicyTrigger()` at appropriate intervals.

Use manual operation only when controlling the timing of DCGM work is an explicit requirement.

Missing calls create monitoring gaps; unnecessarily frequent calls waste resources.

The dynamic linker selects the `libdcgm.so.4` loaded by an embedding process. Search paths such as `LD_LIBRARY_PATH` can change which library that process and its children load. Use compatible runtime and development packages and verify the resolved library when diagnosing a version mismatch.

Next Steps

- [Build Your First DCGM API Clients](#) reviews the installed SDK samples and provides buildable standalone and embedded C clients.
- [C/C++ API](#) covers linking and the public C/C++ interfaces.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.